

Sections 15–17: Detection Tuning, Incident Response, Documentation

15. Detection Tuning

15.1 False Positives — how to actually reduce them (not just define them)

A false positive isn't "an alert I don't like" — it's an alert whose triggering condition is confirmed benign after investigation. Track every one:

| Alert | Rule ID | Reason FP | Tuning Action Taken | Date |

Never silently ignore an alert without logging why — undocumented tuning is how real SOCs slowly go blind to their own environment.

15.2 Whitelisting (do this narrowly, never by disabling the rule entirely)

Bad tuning: disable rule 10.5 (brute force) because your vulnerability scanner trips it. Good tuning: exclude the scanner's specific source IP from that rule only:

```
<rule id="100050" level="0">

  <if_sid>5712</if_sid>

  <srcip>192.168.56.99</srcip> <!-- known vuln scanner -->

  <description>Whitelisted: internal vuln scanner brute-force noise</description>

</rule>
```

This preserves detection for every other source while suppressing the one known-noisy one — the rule still works against a real attacker.

Second example — closing a gap flagged, not fixed, when the Sophos layer was added: detection 10.19's own false-positive note in [03-Detection-Engineering.md](#) warns that a WAN-facing Sophos deny-burst rule "will fire constantly on the WAN side... you'll drown in noise from the public internet." Here's the actual fix rather than leaving that as an open problem:

```
<rule id="110112" level="0">
```

```
<if_sid>110110</if_sid>
```

```
<field name="sophos.srczone">WAN</field>
```

```
<description>Suppressed: routine internet background-radiation scanning on WAN interface</description>
```

```
</rule>
```

This demotes the WAN-side alert to level 0 (logged, not queued for triage) while leaving the *same underlying rule* fully active for LAN-side sources — which is the case that actually matters (an internal host port-scanning your Sophos LAN interface is a real finding; Shodan probing your WAN IP for the millionth time is not). The `sophos-fwrule` decoder in [08-Sophos-Firewall-Integration.md](#) §4.3 now captures `srczone` (extended in the same audit pass that flagged this gap), so this rule works against the decoder as delivered — verify the exact zone name ("WAN" vs. a custom zone name you may have configured) against your own `wazuh-logtest` output rather than assuming it matches the default.

15.3 Noise Reduction Strategies

- **Aggregate before alerting**, not after — a `frequency/timeframe` rule (like 10.5) is inherently quieter than alerting on every single failed login.
- **Layer severity, don't binary-alert** — Wazuh's 0-15 level scale exists so low-signal events can sit in the index (for hunting) without hitting the analyst's active queue.
- **Baseline before ruling** — run the environment for a few days, note what's "normal" (e.g., which service accounts routinely do interactive-looking logons), before writing tight exclusion logic.

15.4 Alert Prioritization

Priority = **Severity × Confidence × Asset Criticality**, not severity alone. A "critical" LSASS-access alert on a disposable Kali VM you own means nothing; the same alert on your SIEM server itself is top priority. In a lab, simulate this by explicitly tagging assets:

Tier 1 (crown jewel): Wazuh/MISP servers

Tier 2: Windows/Linux endpoints

Tier 3: Attacker VM (expected noise source, exclude from alert priority entirely)

15.5 Risk Scoring (simple, defensible model for a lab-scale writeup)

$$\text{risk_score} = (\text{rule_level} / 15) * 0.5 + (\text{asset_tier_weight}) * 0.3 + (\text{ioc_match} ? 1 : 0) * 0.2$$

Document this formula explicitly in your report — a reviewer respects a stated, simple, explainable model far more than an unexplained "AI-powered risk score" claim you can't defend in an interview.

16. Incident Response (NIST-aligned 4-phase cycle, applied to each simulated incident)

16.1 Containment

Short-term: isolate the host from the network without powering it off (preserves memory/forensic state).

Wazuh active response example — network isolation via host firewall

```
sudo iptables -A INPUT -s 192.168.56.10 -j DROP
```

```
sudo iptables -A OUTPUT -d 192.168.56.10 -j DROP
```

Windows-side isolation (if agent-initiated):

```
New-NetFirewallRule -DisplayName "IR-Isolate" -Direction Outbound -Action Block  
-RemoteAddress Any
```

Long-term containment: revoke compromised credentials, disable affected accounts, block confirmed C2 IPs at the firewall/DNS layer.

Perimeter-level containment (new — the mechanism referenced but not shown when detection 10.18 was written): unlike the endpoint isolation above, Sophos is both the sensor *and* an enforcement point, so containment can happen at the same device that raised the alert:

1. **Manual (fast, for a single confirmed-bad IP):** Sophos WebAdmin → **Firewall** → **Add Rule** → source = the offending IP → action = Deny → place above any broader allow rule it would otherwise match. Document the rule name/ID in your incident report (12.9 template) so it's traceable back to this specific incident, not left as an unexplained permanent rule six months later.
2. **API-driven (if you want to demonstrate automation, optional):** Sophos exposes a REST API (<https://<device>:4444/webconsole/APIController>) that can push a firewall rule programmatically — worth a mention in your report's "future work" if you don't have time to build it out, since manual blocking is a legitimate, defensible choice for a lab-scale project and doesn't need to be automated to be correct.

Important scoping note: blocking at Sophos stops *future* connections from that source — it does not undo anything already inside your network. Perimeter containment is a complement to endpoint containment (above), never a substitute for it, and this is worth stating explicitly in your report since it's a real, common misunderstanding (people sometimes treat "I blocked it at the firewall" as equivalent to "I contained the incident," which it isn't if the attacker already has a foothold inside).

16.2 Eradication

- Remove identified persistence (scheduled tasks, services, Run keys — from Section 12.7's sweep).
- Remove dropped malware/tooling (delete after hashing/copying for evidence).
- Patch the initial-access vulnerability if identified (e.g., the web shell's upload flaw in 10.16).

16.3 Recovery

- Restore host from a known-clean snapshot (this is why Section 6.1's snapshot discipline matters) rather than trusting a "cleaned" live host.
- Reset all credentials active on the host during the incident window.
- Re-enable network access only after confirming no persistence remains (re-run the Section 12.7 sweep clean).

16.4 Lessons Learned

Every simulated incident gets a short retro, even in a solo lab:

Lessons Learned: [Incident Title]

What worked: [e.g., detection fired within X seconds]

What didn't: [e.g., password spraying evaded per-account threshold — Section 11.5 gap]

Detection gaps identified:

Tuning changes made as a result:

Would this have been caught faster with [tool/process]?

This is the single most interview-relevant artifact in the whole project — "tell me about a time you found a gap in your own detections" is a real question, and this gives you a true, specific answer instead of a generic one.

17. Documentation (templates — fill per your actual build)

17.1 README.md

Already built as this repo's root file — Sections 1-5 plus links to all subsequent docs.

17.2 Architecture Diagram

ASCII versions are in [README.md](#) Section 2; redraw cleanly in draw.io/diagrams.net, export as PNG, save to [Architecture/architecture.png](#), embed in README.

17.3 Project Report (full academic/technical structure)

1. Abstract

2. Objective & Scope

3. Architecture (diagram + log flow)

4. Tools & Justification

5. Implementation (condensed installation steps)

6. Detection Engineering (link to 03-Detection-Engineering.md)

7. Testing & Validation (Section 11 results table + screenshots)

8. Threat Hunting Findings (Section 13 results)

9. Results & Metrics (table below)

10. Limitations

11. Conclusion & Future Work

Metrics table (fill from your actual test runs): | Metric | Value | How Measured | |---|---|---| |
Mean time to alert | | Simulated action timestamp vs. alert timestamp | | Detection coverage | | #
techniques detected / # attempted (Section 11.8 table) | | False positive rate | | Section 15.1 FP
log / total alerts | | MISP enrichment hit rate | | Enriched alerts / IOC-eligible alerts |

17.4 Installation Manual

[02-Installation-and-SIEM-Setup.md](#) (endpoint + SIEM + MISP) and [08-Sophos-Firewall-Integration.md](#) (perimeter) together form the complete installation manual — two files because the perimeter layer has genuinely different prerequisites (physical hardware handling, network isolation decisions) that don't belong mixed into the VM-only steps.

17.5 SOC Runbook ([Runbooks/soc-runbook.md](#))

Daily analyst checklist: check dashboard alert queue → triage by priority (15.4) → investigate per 12.9 template → escalate/close → log FP per 15.1.

17.6 Detection Runbook ([Runbooks/detection-runbook.md](#))

For each detection in Section 10: when it fires, exact steps to take (this is literally Section 10's "Investigation Steps" + "Recommended Response" columns, reformatted as a per-rule quick-reference card).

17.7 Incident Report Template

Use Section 12.9's template, one file per real/simulated incident, saved to [Incident-Response/](#).

17.8 Threat Hunting Report Template

Hunt: [Hypothesis title]

Date:

MITRE Technique:

Query used:

Data source:

Findings: [nothing found / anomaly found — describe]

Follow-up action: [new detection rule authored? tuning? closed as baseline-normal?]

Continue to: [07-Advanced-Challenges-and-Roadmap.md](#)